

Assembleia 05 — Classes e Recomeço Variado

Gerada a partir de feedback real de jogadores externos, verbatim do Tiago: **“Precisamos do sistema de classes... uma magica, uma ladina ou de distancia física, e um guerreiro de combate próximo... o nome tem que ser único por personagem”** e **“O sistema de recomeço de história não reinicia por completo... tem que ser diferente, em diálogos e situações”**. Duas frentes distintas, cada uma com o método completo — consulta individual, síntese, finalistas, votação.

Nota de escopo, respondida antes de qualquer coisa: nada da Assembleia 04 ficou obsoleto. O conceito de arquétipo (personagem primeiro, com ajuste ± 1) se torna concreto aqui — vira as 3 classes — em vez de ser substituído. E a prioridade do orçamento de token fica **mais** forte, não mais fraca: agora ela protege três coisas, não duas (Seção 5).

FRENTE A — Sistema de classes

1. Baseline

Três classes, conforme o pedido: **Mágica**, **Ladina/Distância Física** (uma classe só, duas formas de nomear a mesma coisa — leitura confirmada pela estrutura da frase do Tiago), **Guerreiro (corpo a corpo)**. O sistema de regras já tem exatamente 3 atributos (CRP/MNT/ALM) e o próprio `attrFromTag` de `src/engine.js` já associa ALM a "magia/mística" e reconhece "astúcia/ardil/investigação" como MNT — a pergunta central desta frente é se as 3 classes mapeiam 1:1 nesses 3 atributos, e como isso se encaixa no que a Assembleia 04 já desenhou (arquétipos pré-definidos com ajuste ± 1).

Pergunta que não decidimos aqui, por instrução direta do Tiago: escopo da unicidade do nome (global vs. por jogador vs. por cenário) — ver Seção 4.

2. Consulta individual

Game System Designer — Proposta central: mapeamento direto, **Guerreiro = CRP primário**, **Ladina/Distância = MNT primário**, **Mágica = ALM primário**. $ALM \leftrightarrow$ Mágica já é literal no código (`attrFromTag`, ação "Magia" em `COMBAT_ACTIONS`). $MNT \leftrightarrow$ Ladina se justifica pelo próprio vocabulário que o `attrFromTag` já reconhece pra MNT — astúcia, ardil, investigação — que é precisamente o registro de um combatente à distância/furtivo, não o de um lutador bruto de corpo a corpo (isso fica pra CRP). Distribuição sugerida, somando 6 (mesmo total do jogador hoje e de cada NPC — `data.js`): primário 3, secundário 2, terciário 1. Isso **não é uma camada nova** — é o mesmo conceito de arquétipo da Assembleia 04, só que com nome e identidade reais em vez de "arquétipo A/B/C". O ajuste ± 1 da Assembleia 04 continua valendo por cima, dentro da classe escolhida.

Narrative Writer — Concorro com o mapeamento. Ponto à parte: os 3 NPCs fixos hoje já ocupam, sem querer, um espaço parecido com as 3 classes novas (Sira é ALM-primária como a Mágica seria; Brennan é MNT-primário como a Ladina seria; Korrin é CRP-primário como o Guerreiro seria) — ver `src/data.js`. Isso corre o risco de o jogador sentir que "já tem um Brennan fazendo esse papel, por que eu preciso escolher igual?". Não acho que precise redesenhar os NPCs por causa disso (fora de escopo, ninguém pediu), mas o texto de apresentação de cada classe deveria deixar claro o papel do herói do jogador como distinto dos companheiros, não redundante.

Backend Engineer — Sem mudança de schema pra classe em si — cabe dentro de `characters.data.jsonb`, que já existe (Documento 02 da série de entregáveis). O que muda o schema de verdade é a unicidade do nome (Seção 4) — isso sim é uma constraint nova, e depende da decisão do Tiago pra saber qual.

Frontend Engineer — `CharacterCreationGate`: 3 cards de classe com resumo de atributo e um preview de HP/MP calculado na hora (fórmula já existe: $10+CRP, 4+ALM \times 2$), depois campo de nome com validação. Se o nome puder colidir (qualquer escopo de unicidade além de "por jogador"), a UI precisa de uma mensagem de erro clara ao tentar salvar — não um "erro genérico".

Code QA Engineer — Recomendo resolver a checagem de nome único por **tentativa de inserção com constraint no banco**, não por uma consulta prévia de disponibilidade — mais simples, e evita ter que criar uma rota especial que bypassa RLS só pra checar se um nome existe (checar "esse nome existe?" globalmente exigiria enxergar linha de outro usuário, o que hoje o RLS não permite por padrão — mais uma razão pra deixar o próprio `UNIQUE` do Postgres rejeitar na inserção e traduzir o erro, como já fazemos com os erros de auth em `src/auth.js`).

AI Master Engineer — Sem posição forte no mapeamento em si. Uma recomendação técnica: a classe escolhida deveria entrar no `SYSTEM_PROMPT` daquela campanha (uma frase — "o jogador é um guerreiro/ladina/mago") pra o mestre narrar de forma consistente com a identidade escolhida. Custo: poucos tokens fixos, mesmo princípio que já uso pra pensar no orçamento (Seção 5).

Infra Engineer — Sem custo de infraestrutura.

Test Engineer — Quero testar as 3 combinações de classe geram HP/MP corretos e que uma tentativa de nome duplicado (qualquer que seja o escopo decidido) mostra erro claro, não trava a tela.

Realtime Multiplayer Engineer — Sem posição forte.

3. Síntese

Convergência quase total no mapeamento 1:1 direto. Única divergência real: o Narrative Writer nota a sobreposição de identidade com os NPCs fixos — não chega a propor uma solução diferente, é uma ressalva sobre a mesma proposta vencedora, não um finalista concorrente de verdade.

4. Finalistas e votação

#	Finalista	Resumo
F1	Mapeamento 1:1 direto	Guerreiro=CRP, Ladina/Distância=MNT, Mágica=ALM — arquétipos da Assembleia 04 com nome real
F2	Ladina/Distância = CRP também	Guerreiro e Ladina dividem CRP primário, diferindo só no secundário
F3	Classes como camada nova	Cada classe com 2 variações internas (ex.: Guerreiro Ofensivo/Defensivo) — 6 combinações, não 3

F4	F1 + redesenho leve dos NPCs	Ajusta Brennan/Sira/Korrin pra não sobrepor as 3 classes do jogador
F5	Classe só de rótulo, sem stat	3 nomes narrativos, atributo inicial continua igual (2/2/2) pra todo mundo

Agente	Voto	Objeção
Game System Designer	F1	Objeta F5 — "classe sem stat diferente não é classe, é só nome".
Narrative Writer	F4 (dissenso)	Quer resolver a sobreposição com os NPCs agora, não só na v2.
Frontend Engineer	F1	Objeta F3 — mais uma camada de escolha que ninguém pediu.
Backend Engineer	F1	Schema mais simples.
AI Master Engineer	F1	Mais simples de injetar no prompt.
Code QA Engineer	F1	Menor superfície nova pra revisar.
Infra Engineer	F1	Sem diferença de custo.
Test Engineer	F1	Mais fácil de testar (3 combinações, não 6).
Realtime Multiplayer Engineer	F1	Sem posição forte.

Resultado: F1 vence 8 a 1.

5. Vencedor — com a mitigação do dissenso

F1, mapeamento 1:1 direto, com a objeção do Narrative Writer incorporada de forma barata: não redesenhar os NPCs (fora de escopo, custo real sem pedido explícito), mas o texto de cada classe no CharacterCreationGate explicita o papel do herói do jogador como distinto dos companheiros — resolve a sobreposição na camada de texto, sem tocar em `data.js`.

FRENTE B — Recomeço com história variada

1. Baseline

Confirmado no código antes de propor qualquer coisa: `freshIntro()` (`src/app.jsx`) devolve **sempre o mesmo texto**, literalmente `window.MWRPG_DATA.scenario.intro` — zero variação hoje, é exatamente o que o Tiago reportou. O pedido cobre dois níveis: a **abertura** (primeira mensagem) e as **situações ao longo da campanha** (não só o primeiro parágrafo) — qualquer solução que só troque a primeira frase não atende ao pedido inteiro.

Restrição real que pesa em toda opção: o orçamento de token da Groq (Assembleia 04) já está no limite, sem mecanismo de proteção implementado ainda.

2. Consulta individual

Narrative Writer — O pedido tem duas partes e quero atender as duas: a abertura *e* as situações que vêm depois. Isso só acontece de verdade se alguma "âncora temática" da campanha (não só a primeira frase) influenciar a narração do mestre ao longo dos 40 turnos — senão "recomeçar" continua sendo psicologicamente a mesma história com uma introdução diferente colada na frente.

AI Master Engineer — Concordo com a leitura do Narrative Writer, mas entro com a ressalva de custo, porque é meu domínio: qualquer opção que gera a abertura via Groq custa ~1.100 tokens **uma vez** por campanha nova (system prompt + instrução de semente, sem histórico ainda) — barato, é menos de 1% do orçamento de uma campanha completa (~114 mil tokens, Assembleia 04). Se a semente também for injetada no `SYSTEM_PROMPT` de cada turno da campanha (pra afetar situações, não só a abertura), isso soma mais ~50-80 tokens × 40 turnos ≈ 2 a 3 mil tokens **por campanha inteira** — ainda pequeno (~3%), mas é custo recorrente, não único, e isso importa porque o mecanismo de proteção de orçamento (Assembleia 04) ainda não está implementado. Minha posição: o custo pequeno é aceitável, mas prefiro não empilhar mais uma fonte de custo recorrente até aquele mecanismo existir de verdade — se formos de uma opção com custo recorrente, que fique combinado que ela também espera o mesmo sinal verde de "não convidar mais gente" que a Assembleia 04 já definiu.

Code QA Engineer — Perguntei se gerar a abertura livremente via LLM fura a disciplina de proveniência do acervo (`docs/ACERVO-PROVENIENCIA.md`) — conclusão, pensando com cuidado: não fura, porque essa disciplina protege o **material de referência curado** (as 6 entradas do acervo), não a narração livre do mestre, que já é gerada por LLM em todo turno normal do jogo, sem vetting linha a linha, e isso já é aceito como o funcionamento padrão do produto. A única exigência real: se a semente da campanha referenciar uma entrada do acervo, tem que vir do `pickAcervoLore` de verdade (as 6 entradas curadas), nunca o modelo inventando uma "fonte" nova por conta própria.

Backend Engineer — Se a semente precisar persistir pra afetar turnos futuros, `campaign_sessions` ganha uma coluna nova (`seed_jsonb`) — mudança aditiva simples, mesmo padrão de "sem migração formal" já em uso no projeto.

Game System Designer — Não mexe na fórmula de regras. Só uma nota: os atributos iniciais continuam vindo só da classe escolhida (Frente A), a semente da campanha não deveria alterar HP/MP/atributos — mantém os dois sistemas independentes.

Frontend Engineer — Se a abertura for gerada por chamada à Groq em vez de montada localmente, o botão "Recomeçar" passa a ter uma espera real (hoje é instantâneo, é só reset local) — precisa de um estado de carregamento. Resolvível, não é bloqueio.

Infra Engineer — Sem custo de infraestrutura — é só mais uma chamada à mesma Function que já existe.

Test Engineer — Quero um teste objetivo: recomeçar duas vezes seguidas e comparar o texto da abertura — tem que ser visivelmente diferente, não só cosmético. Isso é mais fácil de verificar em qualquer opção que gere texto via LLM do que numa lista estática fixa (onde "diferente" pode, na prática, ainda estar reciclando as mesmas 6 ou 8 frases decoradas).

Realtime Multiplayer Engineer — Sem posição forte.

3. Síntese

Convergência real em quase tudo: todos concordam que "situações", não só "abertura", é o pedido de verdade; todos concordam que gerar via Groq é tecnicamente simples. **A única divergência real é o AI Master Engineer questionando o *timing* do custo recorrente** — não o mecanismo em si — dado que a proteção de orçamento da Assembleia 04 ainda não foi implementada.

4. Finalistas e votação

#	Finalista	Resumo	Custo estimado
F1	Pool estático (sem LLM)	N aberturas escritas à mão pelo Narrative Writer, sem repetir a última usada; situações ao longo da campanha continuam como hoje	Zero
F2	Semente de campanha persistente, gerada por Groq	Gancho + entrada do acervo + variável sensorial sorteados sem repetir consecutivo, renderizados em prosa (1 chamada Groq) e injetados no SYSTEM_PROMPT da campanha inteira — abertura *e* situações variam	~1.100 tokens únicos + ~2-3 mil/campanha recorrente
F3	Só abertura gerada por Groq, sem persistir	Mesma geração da F2, mas a semente não afeta os turnos seguintes	~1.100 tokens únicos, zero recorrente
F4	Híbrido processual sem LLM	Pool de ganchos escritos com 2-3 variáveis sorteadas de listas curtas (quem fala primeiro, clima, detalhe sensorial)	Zero

F5	Adiar	Não mexe em nada até o mecanismo de orçamento da Assembleia 04 estar pronto e testado	Zero (por enquanto)
----	-------	---	---------------------

Agente	Voto	Justificativa / objeção
AI Master Engineer	F3 (dissenso)	Aceita o custo único e pequeno; objeta ao custo recorrente da F2 enquanto a proteção de orçamento não existe de verdade.
Narrative Writer	F2	Só F2 atende o pedido inteiro — abertura *e* situações; as outras deixam a campanha "igual por dentro" depois da primeira frase.
Backend Engineer	F2	Schema simples, mudança aditiva de uma coluna.
Game System Designer	F2	Sem objeção mecânica.
Frontend Engineer	F2	Estado de carregamento é resolvível.
Code QA Engineer	F2	Com a exigência de que a semente só referencie o acervo curado de verdade, nunca uma fonte inventada.
Infra Engineer	F2	Sem custo de infra.
Test Engineer	F2	Mais fácil de provar que a variação é real.
Realtime Multiplayer Engineer	F2	Sem posição forte.

Resultado: F2 vence 8 a 1 (AI Master Engineer dissente pra F3).

5. Vencedor — com a mitigação do dissenso

F2, "semente de campanha persistente, gerada por Groq", com a objeção do AI Master Engineer incorporada, não descartada — usando o mesmo princípio já estabelecido na Assembleia 04:

- Implementar F2 (custo recorrente pequeno, ~3% de uma campanha) **em paralelo** com o resto do trabalho.
- Mas o **lançamento** — abrir isso pra mais jogadores de teste — fica atrás do mesmo portão que a Assembleia 04 já definiu: nenhum convite novo até o mecanismo de proteção de orçamento estar pronto e testado em produção. F2 não pula essa fila — só entra na fila junto com personagem/classes, não à frente do orçamento.

- Medir o custo real da primeira geração de semente (tokens de verdade, não a estimativa desta assembleia) assim que implementado, pra confirmar ou corrigir o número de ~1.100 tokens antes de qualquer decisão futura de expandir.

Como isso reconcilia com a Assembleia 04 — respondendo direto

Nada ficou obsoleto. Dois ajustes, os dois no sentido de reforçar, não substituir:

- 1 **Arquétipos** → **Classes**: o que a Assembleia 04 chamou de "arquétipos pré-definidos com ajuste ± 1 " ganha nome e identidade real aqui — Guerreiro/Ladina/Mágica **são** os arquétipos, mapeados 1:1 nos 3 atributos já existentes. O mecanismo de ajuste ± 1 segue valendo, agora dentro de cada classe nomeada.
- 2 **Orçamento de token**: a prioridade fica **mais** alta, não mais baixa — agora ela é o portão de lançamento de **três** coisas (personagem/classes, recomeço variado, e qualquer convite novo de teste), não só duas. O AI Master Engineer levantou essa mesma bandeira nas duas frentes desta assembleia, de forma consistente com a posição que já tinha na Assembleia 04.

Questão em aberto — decisão do Tiago, não da assembleia

Escopo da unicidade do nome do personagem. Por instrução direta dele ("se ficar ambíguo, me pergunte antes de decidir"), não resolvemos isso por votação — os agentes concordam na técnica (constraint no banco, erro traduzido ao tentar salvar — Seção 4, Frente A), mas o **escopo** é uma decisão de produto real, com trade-off genuíno:

Escopo	A favor	Contra
Global (nenhum outro jogador pode ter o mesmo nome, nunca)	Identidade forte — o nome é único no mundo de Ys, como um personagem "de verdade"	Jogador que chega depois pode não conseguir o nome que quer, mesmo sendo natural (ex.: dois jogadores diferentes, ambos querendo "Aldric")
Por cenário (único dentro de "A Coroa Enterrada de Ys", mas cenários futuros teriam seu próprio espaço de nomes)	Meio-termo — forte dentro do mundo atual, sem travar pra sempre se o jogo crescer pra mais cenários	Só existe 1 cenário hoje, então na prática se comporta como global até isso mudar — o benefício é só teórico agora
Por jogador (só não pode repetir dentro da própria conta)	Nunca frustra ninguém — dificilmente alguém tenta criar 2 personagens com o mesmo nome na própria conta	Não é unicidade de verdade — hoje cada conta só tem 1 personagem por vez, então essa constraint quase nunca dispara; não entrega a "identidade única" que parece ser a intenção por trás do pedido

Fico no aguardo dessa resposta antes de implementar a constraint — tudo o resto do plano pode avançar sem ela estar decidida.

Aguardando aprovação do Tiago

Nenhuma linha de código deste plano até o sinal explícito dele — só este documento não depende de aprovação, é processo, não produto.